

02 — Carte des fonctionnalités

Légende criticité : ● = à maîtriser absolument · ● = important · ● = secondaire.

Authentification ●

- **Rôle** : tous.
- **Front** : Frontend/login.html , Frontend/script/login.js , Frontend/register.html , Frontend/script/register.js , Frontend/script/config.js (token en localStorage), Frontend/script/auth_guard.js (garde de page).
- **Backend** : API/admin/users.go (Login , Inscription), API/auth/jwt.go (JWT + middlewares).
- **Tables** : utilisateur , log_connexion .
- **Endpoints** : POST /admin/login , POST /auth/inscription , POST /auth/check-email , POST /auth/forgot-password , POST /auth/reset-password .

Gestion des utilisateurs ●

- **Rôle** : Administrateur.
- **Front** : Frontend/admin_users.html , Frontend/script/admin/user.js (pagination 10/page).
- **Backend** : API/admin/users.go , API/bdd/userReq.go .
- **Tables** : utilisateur , documents_legaux .
- **Endpoints** : GET /admin/users , GET /admin/users/{id} , GET /admin/users/search , GET /admin/users/role/{role} , POST /admin/users/add , PUT /admin/users/modify/{id} , DELETE /admin/users/delete/{id} , PUT /admin/users/validate|refuse|ban/{id} .

Rôles & permissions ●

- **Rôles** (enum utilisateur.role) : Utilisateur , Pro , Salarié , Administrateur .
- **Backend** : API/auth/jwt.go → VerifyTokenMiddleware (vérifie le JWT) et VerifyRoleMiddleware(next, roles...) (vérifie le rôle). Rôle mis dans context (userRole).
- **Front** : redirection par rôle dans Frontend/script/login.js ; garde côté page Frontend/script/auth_guard.js (checkSession(role)).

Annonces (dons / ventes) ●

- **Rôle** : Utilisateur (crée), Pro (achète/sponsorise), Admin (valide).
- **Front** : Frontend/annonceAll.html + Frontend/script/annonces/annonceAll.js , Frontend/oneAnnonce.html + oneAnnonce.js , Frontend/espClient.html + Frontend/script/dashClient.js , admin : Frontend/script/admin/annonce.js .
- **Backend** : API/admin/annonce.go , API/bdd/annonceReq.go .
- **Tables** : annonce , categorie .
- **Endpoints** : GET /api/annonces , GET /api/annonces/all , GET /mes-annonces , POST /admin/annonces/add , PUT /admin/annonces/modify/{id} , PUT /admin/annonces/validate|refuse/{id} , DELETE /admin/annonces/delete/{id} , POST /api/annonces/vendre , POST /api/payment-annonce .

Catégories / matériaux ●

- **Rôle** : Admin (gère), tous (filtre).
- **Front** : filtre dynamique `Frontend/script/annonces/annonceAll.js` (`buildCategoryFilter`).
- **Backend** : `API/admin/categorie.go` , `API/bdd/categorieReq.go` .
- **Table** : `categorie` (libellés : Textile, Bois, Plastique, Métal).
- **Endpoints** : `GET /admin/categories` , `POST /admin/categories/add` , `DELETE /admin/categories/delete/{id}` .

Conteneurs & casiers (logistique) ●

- **Rôle** : Admin (gère), Utilisateur (dépose/récupère), Pro (récupère).
- **Front** : `Frontend/admin_conteneurs.html` + `Frontend/script/admin/box.js` (+ **rapport logistique PDF** via jsPDF).
- **Backend** : `API/admin/box.go` , `API/bdd/boxReq.go` .
- **Tables** : `conteneur` , `box` , `box_conteneur` , `depot_box` , `historique_conteneurs` .
- **Endpoints** : `GET /api/admin/conteneurs` , `GET /api/admin/conteneur/{id}/boxes` , `POST /api/admin/conteneur/create` , `POST /api/admin/box/add` , `PUT /api/admin/box/update` .

Réservation / dépôt / retrait (hardware simulé) ●

- **Front** : `Frontend/simulateur.html` (simulateur dépôt/retrait), `Frontend/script/dashClient.js` (boxes user).
- **Backend** : `API/admin/box.go` (deposit/collect/reserve).
- **Tables** : `depot_box` , `historique_conteneurs` , `box` .
- **Endpoints** : `POST /api/box/reserve` , `POST /api/box/deposit` , `POST /api/box/collect` , `POST /api/boxes/valider-retrait` , `POST /api/hardware/simulate-deposit` , `POST /api/hardware/simulate-withdrawal` , `GET /api/user/boxes` , `GET /api/user/pickups/{id}` .

Commandes / transactions / commissions ●

- **Rôle** : Utilisateur (achète), Admin (suivi finances).
- **Front** : `Frontend/admin_finances.html` + `Frontend/script/admin/finance.js` .
- **Backend** : `API/admin/orderReq.go` , `API/admin/stripe.go` , `API/bdd/order.go` .
- **Tables** : `order` , `paiement` , `document` (factures).
- **Commission** : 5 % — voir `API/admin/stripe.go` (`(unitAmount * 5) / 100 / prix * 0.05`).
- **Endpoints** : `GET /admin/finance/overview` , `GET /admin/finance/transactions` , `GET /api/user/payment-history` , `GET /api/user/achats` .

Upcycling Score (impact éco) ●

- **Rôle** : Utilisateur.
- **Front** : `Frontend/espClient.html` + `Frontend/script/dashClient.js` (`loadEcoScore`).
- **Backend** : `API/admin/annonce.go` (`GetEcoStatsHandler`), `API/bdd/annonceReq.go` (`GetUserEcoStats`), `API/bdd/boxReq.go` (`CalculateAndAddScore`).
- **Table** : `utilisateur.score` , `upcycling_score` .
- **Endpoints** : `GET /api/user/stats` , `GET /api/user/ecostats` .
- **Calcul** : `gainScore = poids_kg × coefficient(matériau)` — coefficients dans `CalculateAndAddScore` (textile 15, métal 10, plastique 8, bois 5, autre 3). ⚠ **À confirmer** : dans le code actuel la variable `materiau` n'est pas remplie par la requête SQL → le coefficient tombe toujours sur `autre` (3.0). (Point d'amélioration honnête à connaître.)

Événements & formations ●

- **Rôle** : Salarié (crée), Utilisateur/Pro (s'inscrit, paie), Admin (valide).
- **Front** : Frontend/evenement.html + Frontend/script/affichageEvt.js , salarié : Frontend/salarie/salarie_events.html + Frontend/script/salarie/event.js , planning : Frontend/script/salarie/planning.js .
- **Backend** : API/admin/event.go , API/bdd/eventReq.go , API/admin/stripe.go (CreateEventCheckoutSession).
- **Tables** : evenement , inscription , ressource_pedagogique .
- **Endpoints** : GET /admin/evenements , POST /admin/evenements/add , PUT /admin/evenements/validate|refuse/{id} , POST /admin/evenements/inscription|desinscription , POST /api/web/checkout/evenement , GET /admin/evenements/inscrits/{id} , GET /admin/evenements/ressources/{id} .
- **Règle métier** : un salarié doit avoir un stripe_account_id pour déposer un event (API/admin/event.go , CreateEvenement).

Articles / actualités

- **Rôle** : Salarié (rédige), Admin (valide), tous (lecture).
- **Front** : Frontend/article.html + Frontend/script/affichageArt.js , salarié : Frontend/script/salarie/article.js .
- **Backend** : API/admin/article.go , API/bdd/articlesReq.go .
- **Table** : article_news (PK id_article) .
- **Endpoints** : GET /admin/articles , POST /admin/articles/add/{action} , PUT /admin/articles/validate|refuse/{id} , GET /admin/articles/salarie/{id} .

Forum

- **Front** : Frontend/forum.html + Frontend/script/forum.js , salarié : Frontend/script/salarie/forum.js .
- **Backend** : API/admin/forum.go , API/bdd/forumReq.go .
- **Tables** : topic_forum , message_forum .
- **Endpoints** : GET/POST /user/forums , GET/POST /user/forums/messages , GET /admin/forum/messages , PUT /admin/forum/messages/moderate/{id} , GET /admin/forum/stats .

Messagerie / chat (WebSocket)

- **Front** : Frontend/script/messagerie/messagerie.js .
- **Backend** : API/admin/chat.go , API/bdd/chatReq.go .
- **Table** : message .
- **Endpoints** : GET /ws/chat (WebSocket), GET /api/chat/conversations , GET /api/chat/history .

Espace Pro (abonnements, projets, sponsoring)

- **Front** : Frontend/espPro.html + Frontend/script/dashPro.js .
- **Backend** : API/route/pro.go , API/admin/stripe.go , API/admin/projet.go , API/admin/etape.go .
- **Tables** : abonnement , plan_abo , projet_pro , etapes_projet .
- **Endpoints** : POST /api/pro/subscribe|upgrade|cancel|portal , GET /api/pro/sync|invoices|projets|etapes , POST /api/pro/projets/create , POST /api/pro/annonces/sponsor .

Back-office admin

- **Front** : Frontend/admin_dashboard.html (+ Frontend/script/admin/dash.js) , admin_users.html , admin_validations.html , admin_finances.html , admin_documents.html , admin_conteneurs.html .

- **Backend** : dossier `API/admin/` .
- **Sécurité** : toutes les routes `GET /admin/*` (hors login) sont protégées par `VerifyTokenMiddleware` / `VerifyRoleMiddleware` .

Notifications

- **Front** : `Frontend/script/salarie/userInfo.js` (`chargerNotifications` , cloche), SDK OneSignal inline dans `espClient.html` / `espPro.html` / `admin_*` (`OneSignal.login(userId)`).
- **Backend** : `API/admin/notifications.go` (`SendPushNotification` , `OneSignal`, `NotifyAllAdmins` , `CreateNotification`).
- **Table** : `notification` .
- **Endpoints** : `POST /admin/notifications/send` , `GET /admin/notifications/user/{id}` , `POST /admin/notifications/user/{id}/read` .

Génération PDF

- **Factures / contrats** (serveur, lib `go-pdf/fpdf`) : `API/admin/pdf.go` (`GenerateInvoicePDF` , `GenerateContractPDF`) → stockés dans le volume `/app/uploads/documents/` , listés dans `Frontend/admin_documents.html` + `Frontend/script/admin/documents.js` .
- **Rapport logistique** (client, lib `jsPDF`) : `Frontend/script/admin/box.js` (`genererRapportLogistique`).
- **Table** : `document` .

Multilingue

- **Front** : `Frontend/script/admin/translate.js` (`t()` , `appliquerTraductions()` , sélecteur de langue en bas à droite), attributs `data-i18n` .
- **Backend** : `API/admin/translation.go` , `API/bdd/translation.go` .
- **Tables** : `translations` (`lang_code`, `msg_key`, `msg_value`), `languages` .
- **Endpoints** : `GET /api/translations?lang=fr` , `GET /api/languages` , `POST /admin/translations/add` .

Paiement Stripe

- **Backend** : `API/admin/stripe.go` (Checkout + Connect + commission 5 %), `API/admin/webhook.go` (`POST /api/stripe/webhook`).
- **Config** : clés en variables d'env `STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` (fallback en dur dans `stripe.go`).
- **Uploads / fichiers** : `API/admin/upload.go` (`SaveUpload` : validation ext + MIME + taille, nom UUID), route `/uploads/*` , volume `uploads_data` .

Déploiement Docker

- **Local** : `docker-compose.yml` (build local).
- **Prod / VM** : `docker-compose-prod.yml` (images `amelbdj/upcycle-backend` , `amelbdj/upcycle-frontend`), derrière un Nginx hôte HTTPS (`https://upcycleconnect.pro`).